

# Initial Exploration of Time-Series Dataset

April 9, 2025

## Step 1: Load and Display the Dataset

### Code

```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv('dataset.csv')

# Display first few rows
print(df.head())
```

### Output: First 3 Rows (Extract)

|   | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | 10    | ... | 1991  |
|---|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|-------|
| 0 | 13    | 29    | 44    | 55    | 61    | 72    | 200   | 96    | 112   | 120   | ... | 21680 |
| 1 | -3980 | -3883 | -3832 | 0     | -3839 | -3788 | -3695 | -3663 | -3669 | 0     | ... | 7130  |
| 2 | -3959 | -4058 | -4131 | -4033 | -4009 | -3941 | -3996 | -4071 | -3971 | -4021 | ... | -254  |

|   | 1992  | 1993  | 1994  | 1995  | 1996  | 1997  | 1998  | 1999  | 2000  |
|---|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 0 | 21692 | 21697 | 21705 | 21711 | 21729 | 21742 | 21746 | 21763 | 21777 |
| 1 | 7150  | 7117  | 7023  | 7064  | 0     | 7027  | 6929  | 6954  | 6867  |
| 2 | -296  | -303  | 0     | -327  | -274  | -358  | -278  | -346  | -299  |

### Explanation:

This initial step loads the dataset `dataset.csv` using pandas and displays the first few rows to inspect its structure. The data appears to be numerical and time-series-like, with columns labeled from 1 to 2000 (possibly representing time points, features, or IDs).

- Row 0 appears to contain positive increasing values — potentially a reference or index sequence.
- Rows 1 and 2 contain both positive and negative values, possibly representing measured signals or transformed data.
- The high dimensionality (2000 columns) suggests this dataset may require dimensionality reduction or feature engineering in later steps.

This preview helps understand the shape and format of the dataset before further processing or visualization.

## Step 2: Generate Column Index Matrix

### Code

```
column_indices = np.arange(len(df.columns)).reshape(-1, 1)
print(column_indices)
```

### Output (Partial View):

```
[[ 0]
 [ 1]
 [ 2]
 ...
[1997]
[1998]
[1999]]
```

### Explanation:

This code creates a matrix of **column indices** from the DataFrame `df`, which contains 2000 columns (from 0 to 1999). The function `np.arange(len(df.columns))` generates a sequence from 0 to 1999, representing the index of each column in order.

The call to `reshape(-1, 1)` transforms this flat array into a 2D column vector with shape (2000, 1) — one row per column index.

#### Why reshape to (n, 1)?

- Many machine learning or signal processing libraries (e.g., scikit-learn, TensorFlow) expect inputs to be 2D arrays where rows represent samples and columns represent features.
- The reshaped structure is especially useful when column indices need to be passed into functions like scalers, encoders, or plotted along time axes.
- In time-series analysis, this may serve as the time vector for plotting or aligning temporal signals across samples.

This indexing step provides a structured and interpretable way to reference features, particularly useful in high-dimensional datasets such as this one with 2000 attributes.

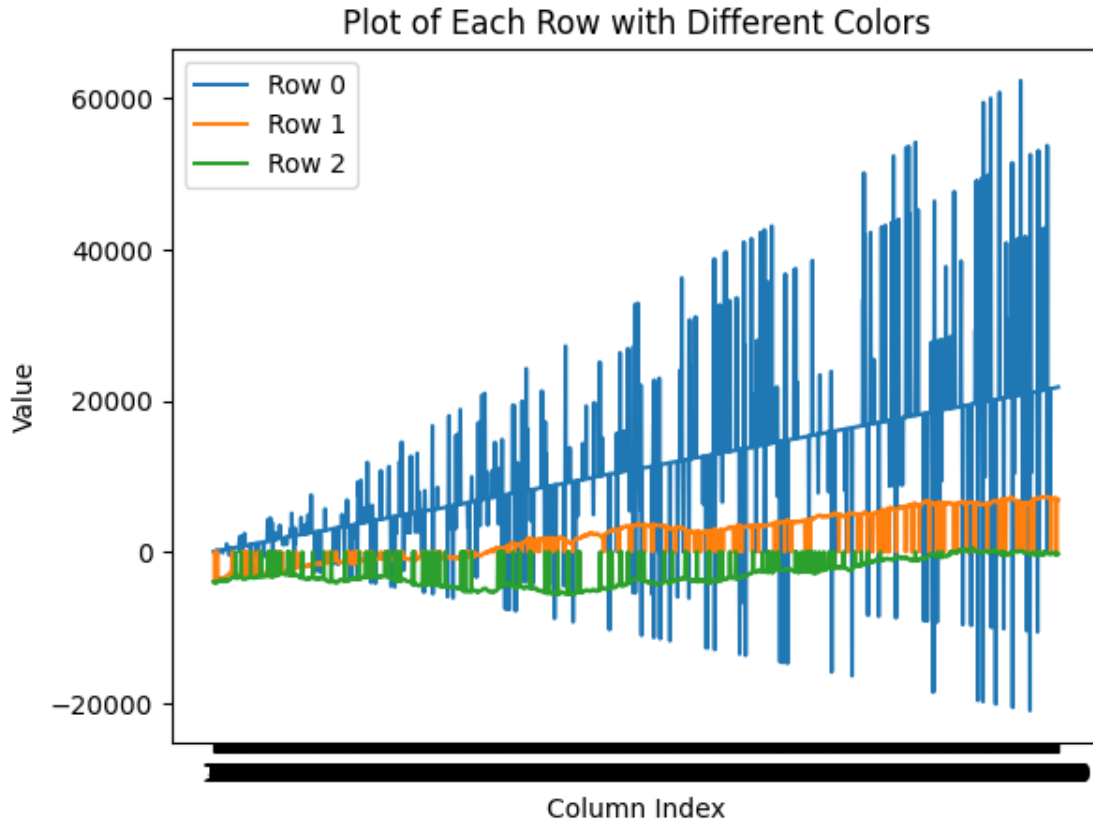
## Step 3: Visualizing Rows as Line Series

### Code

```
import matplotlib.pyplot as plt

# Plot each row with a different color
for i in range(df.shape[0]):
    plt.plot(df.columns, df.iloc[i], label=f'Row-{i}')

# Add labels and a title to the plot
plt.xlabel("Column-Index")
plt.ylabel("Value")
plt.title("Plot of Each Row with Different Colors")
plt.legend()
plt.show()
```



### Explanation:

This step visualizes each row of the dataset as a separate line on a single 2D plot. Since the dataset has a high number of columns (2000), this visualization provides a way to observe **patterns across features** for individual data samples.

- The loop iterates through each row in the DataFrame using `df.shape[0]`, which counts the number of samples (rows).
- For each row, `plt.plot(df.columns, df.iloc[i])` plots that row's values across all column indices. Each row is rendered with a unique color and labeled accordingly in the legend.
- `plt.xlabel` and `plt.ylabel` provide context for the axes: column indices on the x-axis (likely time points or feature positions) and actual recorded values on the y-axis.
- The title and legend make the chart easier to read and interpret, especially when comparing trends across rows.

### Use Cases and Insights:

- This kind of plot is especially useful in **time-series analysis**, where each row could represent a temporal measurement (e.g., sensor readings over time).
- It helps identify **signal structure**, **alignment**, **drift**, or **outliers** between different observations.
- Variability between rows becomes immediately visible — for example, if some rows fluctuate more widely, have sudden jumps, or flatline in comparison to others.

This step is an important part of **exploratory data analysis (EDA)** in high-dimensional numerical datasets. By visualizing rows against columns, it gives a quick and intuitive understanding of the data's internal behavior before more complex modeling or transformation.

## 2.2 Data Cleaning and Preprocessing

### 2.2.1 Why Data Cleaning Matters in Preprocessing

Data cleaning is a crucial step in the preprocessing stage of any machine learning workflow. Raw datasets often include issues like missing entries, noise, or inconsistencies that can hinder the performance of predictive models. By cleaning the data, we improve its quality and reliability, making it easier for models to identify meaningful trends. This ultimately enhances model accuracy, ensures better generalization to new data, and results in more dependable predictions.

### 2.2.2 Techniques Used for Data Cleaning

For cleaning the dataset, we applied an outlier detection method based on the Interquartile Range (IQR). Any data point below:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR} \quad \text{or above} \quad \text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}$$

was flagged as an outlier.

Instead of removing these extreme values, we opted to replace them with the average of their immediate neighbors within the same column. This technique has the advantage of:

- Preserving the overall shape and structure of the data
- Minimizing distortion caused by outliers
- Maintaining continuity between neighboring values

This balanced approach results in a dataset that is more robust, with reduced variance due to noise, while still retaining the underlying patterns needed for effective modeling and analysis.

## Step 4: IQR-Based Outlier Handling with Neighbor Averaging

### Code

```
import pandas as pd
import numpy as np

def clean_outliers_with_neighbor_average(dataframe):
    cleaned_df = dataframe.copy()

    # Loop through numeric columns only
    for column in cleaned_df.select_dtypes(include=[np.number]).columns:
        Q1 = cleaned_df[column].quantile(0.25)
        Q3 = cleaned_df[column].quantile(0.75)
        IQR = Q3 - Q1

        lower_limit = Q1 - 1.5 * IQR
        upper_limit = Q3 + 1.5 * IQR

        # Check each value in the column
        for idx in range(len(cleaned_df)):
            value = cleaned_df.loc[idx, column]
            if value < lower_limit or value > upper_limit:
                prev_val = cleaned_df[column].iloc[idx - 1] if idx > 0 else np.nan
                next_val = cleaned_df[column].iloc[idx + 1] if idx < len(cleaned_df) - 1 else np.nan
                mean_val = np.nanmean([prev_val, next_val])
                cleaned_df.loc[idx, column] = mean_val

    return cleaned_df

# Apply the function to your DataFrame
df_cleaned = clean_outliers_with_neighbor_average(df)
```

### Explanation:

This function performs outlier detection and correction across all numerical columns of a dataset using a combination of the **Interquartile Range (IQR)** method and **neighbor-based smoothing**. Here's a breakdown of the approach:

- The function begins by making a deep copy of the original DataFrame to avoid modifying it in place.
- For each numeric column:
  - The first (Q1) and third (Q3) quartiles are computed.
  - The Interquartile Range (IQR) is calculated as  $Q3 - Q1$ .
  - Values are flagged as outliers if they lie outside the range:  $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ .
- Rather than dropping outliers, the algorithm replaces them with the **mean of their immediate neighbors** within the same column:
  - For value at index `idx`, it takes the previous value (`idx - 1`) and the next value (`idx + 1`).
  - If either neighbor is missing (i.e., edge cases), `np.nanmean` gracefully handles the averaging.

### Why This Matters:

- Removing outliers can distort dataset size and introduce discontinuities.
- Imputing outliers with neighboring averages maintains the natural structure and trend of the data.
- This method reduces the influence of extreme values while avoiding hard deletion or overly synthetic replacements (like global means).

The resulting DataFrame, `df_cleaned`, is more robust for downstream analysis, especially in sensitive models like regression, where outliers can disproportionately affect coefficients or error metrics.

### 2.2.3 Comparing the Original and Cleaned Data

To visualize the effect of outlier handling, this section compares the first row of the original dataset (`df`) with the corresponding row in the cleaned dataset (`df_cleaned`). This is achieved through a grouped bar chart, where each feature (column) is represented by two bars:

- One bar for the original value
- One bar for the cleaned value

By placing these bars side by side for each feature, we can easily identify which values were modified during the cleaning process. Columns where outliers were detected and corrected will show noticeable differences between the original and cleaned values, while unchanged columns will appear identical.

#### **Purpose and Benefits of the Comparison Chart:**

- It provides a direct, visual comparison of the row before and after cleaning.
- It helps validate that only outliers were changed, and most of the data was preserved.
- It increases transparency in preprocessing by illustrating how the cleaning method affects real data.

This kind of visualization is especially helpful when presenting your preprocessing steps to stakeholders or collaborators, as it makes the abstract concept of “outlier smoothing” concrete and interpretable.

In the chart above, each x-axis label represents a column (or feature), and the y-axis shows the actual value. Colored bars distinguish between the original data and cleaned data. Any significant difference between paired bars highlights the effect of the IQR-based correction method.

If the row contains few or no outliers, the bars will appear nearly identical. However, for rows that had multiple outliers, the corrected values tend to be closer to their neighboring values and exhibit less extreme behavior, resulting in a smoother and more reliable dataset for further analysis or modeling.

## Step 5: Visual Comparison of First Row Before and After Cleaning

### Code

```
import matplotlib.pyplot as plt
import numpy as np

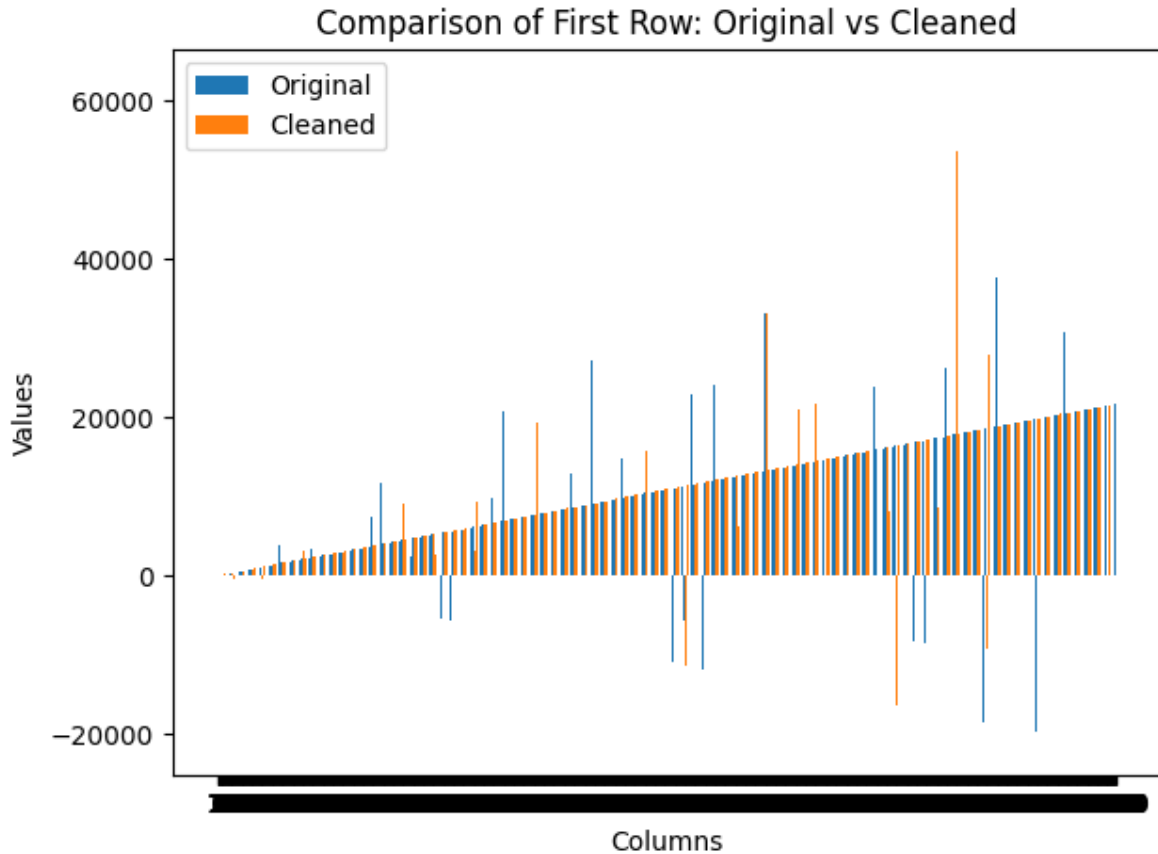
# Select the first row from the original and cleaned DataFrames
original_row = df.iloc[0]
cleaned_row = df_cleaned.iloc[0]

# Define labels and positions for the bars
column_labels = original_row.index
x_pos = np.arange(len(column_labels))
bar_width = 0.35

# Create the bar plot
fig, ax = plt.subplots()
ax.bar(x_pos - bar_width / 2, original_row, width=bar_width, label='Original')
ax.bar(x_pos + bar_width / 2, cleaned_row, width=bar_width, label='Cleaned')

# Customize the chart
ax.set_xlabel('Columns')
ax.set_ylabel('Values')
ax.set_title('Comparison of First Row: Original vs Cleaned')
ax.set_xticks(x_pos)
ax.set_xticklabels(column_labels)
ax.legend()

plt.tight_layout()
plt.show()
```



### Explanation:

This plot provides a **side-by-side visual comparison** of the first row in the dataset before and after applying outlier cleaning. By rendering both the original and cleaned values as **grouped bars** for each column, this visualization makes it immediately obvious which data points were modified.

- The x-axis represents the feature or column index.
- Each group contains two bars:
  - One for the original value (blue or left bar)
  - One for the cleaned value (orange or right bar)
- The y-axis displays the actual magnitude of each value in the row.

### Benefits of the Visualization:

- It highlights where the IQR-based outlier detection method identified unusually high or low values.
- Any bar pair with a noticeable difference reflects an outlier that was corrected using the neighbor-averaging strategy.
- This targeted approach maintains the shape of the signal while addressing instability caused by extreme values.

### Interpretation:

- In columns where the two bars are nearly identical, no cleaning was necessary — the values fell within the IQR range.
- In columns with visible differences, the original outlier was replaced with the average of adjacent non-outlier values.
- If the first or last element of the row was an outlier, only one neighbor may be used (edge case), and this is handled gracefully using `np.nanmean()`.

This visualization validates that the data cleaning operation was selective and non-destructive — a crucial insight when working with sensitive or high-stakes datasets. It also builds trust in the preprocessing pipeline by making the modifications transparent and explainable.

## 2.3 Building a Regression Model from Scratch

### 2.3.1 Data Preparation

To begin building our regression model, we first prepared the dataset. The input features, denoted as  $X$ , were obtained by selecting all columns except the final one, which was chosen as the target variable  $y$ . To include an intercept (or bias) term in the model, we added a column of ones to the feature matrix. This addition allows the model to learn a non-zero intercept and improves its flexibility during training.

### Defining the Cost Function

To evaluate how well our model performs, we used the **Mean Squared Error (MSE)** as the cost function. MSE computes the average of the squared differences between predicted values and actual target values. Lower MSE indicates more accurate predictions, which is the goal of regression modeling.

The cost function is defined mathematically as:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Where:

- $m$  is the number of training samples
- $h_{\theta}(x)$  is the predicted output using parameters  $\theta$
- $y^{(i)}$  is the actual target value

### Gradient Descent Optimization

To minimize the cost function and find the optimal parameters  $\theta$ , we applied the **gradient descent algorithm**. This iterative method updates model parameters in the direction that reduces the cost, based on the calculated gradient.

The gradient descent update rule is:

$$\theta := \theta - \alpha \cdot \frac{1}{m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

Where  $\alpha$  is the learning rate, controlling how large each step is during parameter updates.

To monitor progress, we tracked the cost function at every iteration using a list called `cost_history`. This helped ensure that the model was converging toward a lower cost.

## Model Training

We began the training process by initializing all model weights  $\theta$  to zero. Two hyperparameters were set before training:

- Learning rate  $\alpha = 0.01$
- Number of iterations = 1000

Using these values, we ran gradient descent, updating the weights in each iteration and recording the cost along the way. After the final iteration, we obtained a trained weight vector  $\theta_{\text{final}}$ , which captured the relationship between the features and the target variable.

## Making Predictions

With the optimized weights, we made predictions on the dataset. This involved computing the `**dot product**` between the extended input matrix (which includes the intercept term) and the learned weight vector:

$$\hat{y} = X \cdot \theta_{\text{final}}$$

The resulting predictions represent the model's output for each training sample, based on the linear relationships discovered during gradient descent.

## Step 6: Implementing Linear Regression from Scratch using Gradient Descent

### Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Prepare features and target
X = df_cleaned.iloc[:, :-1].values
y = df_cleaned.iloc[:, -1].values

# Feature normalization (optional but recommended)
X_mean = np.mean(X, axis=0)
X_std = np.std(X, axis=0)
X = (X - X_mean) / X_std

# Add intercept (bias) term
X_b = np.c_[np.ones((X.shape[0], 1)), X]

# Cost function
def compute_cost(X, y, theta):
    m = len(y)
    predictions = X.dot(theta)
    cost = (1 / (2 * m)) * np.sum((predictions - y) ** 2)
    return cost

# Gradient Descent
def gradient_descent(X, y, theta, learning_rate, iterations):
    m = len(y)
    cost_history = np.zeros(iterations)

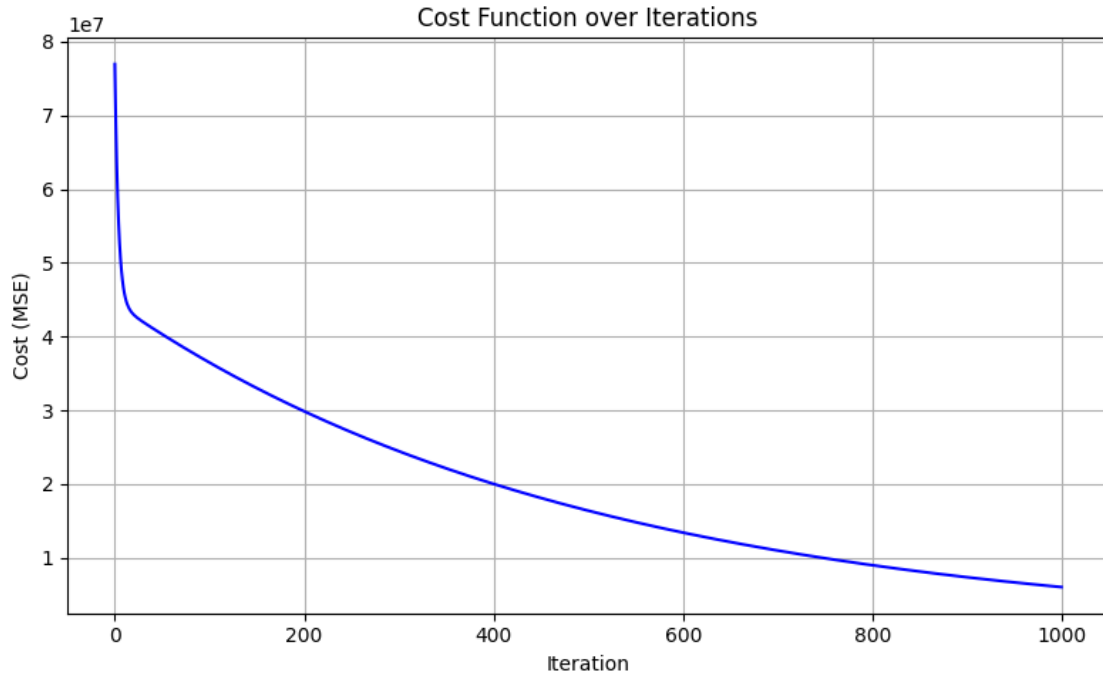
    for i in range(iterations):
        predictions = X.dot(theta)
        gradients = (1 / m) * X.T.dot(predictions - y)
        theta -= learning_rate * gradients
        cost_history[i] = compute_cost(X, y, theta)

    return theta, cost_history

# Initialize parameters
theta_initial = np.zeros(X_b.shape[1])
learning_rate = 0.001 # Reduced learning rate
iterations = 1000

# Train the model
theta_final, cost_history = gradient_descent(X_b, y, theta_initial, learning_rate, iterations)

# Plot cost history
plt.figure(figsize=(8, 5))
plt.plot(range(len(cost_history)), cost_history, color='blue')
plt.title('Cost-Function-over-Iterations')
plt.xlabel('Iteration')
plt.ylabel('Cost-(MSE)')
plt.grid(True)
plt.tight_layout()
plt.show()
```



### Explanation:

This code implements **linear regression from scratch** using **gradient descent**, without relying on any high-level machine learning libraries. Here's a breakdown of the full process:

#### 1. Feature Preparation:

- The input features  $X$  are selected from all columns except the last.
- The target  $y$  is the last column, which we aim to predict.
- Normalization is applied to each feature to ensure all values lie on a comparable scale. This helps gradient descent converge faster and prevents features with larger values from dominating.
- An intercept term (a column of ones) is added to  $X$  to allow the model to learn a bias term.

#### 2. Cost Function:

- The cost is computed using **Mean Squared Error (MSE)**, a standard metric for regression tasks.
- The cost represents how far off the model's predictions are from the actual values.

#### 3. Gradient Descent Training:

- The algorithm iteratively adjusts weights to reduce the cost function.
- On each iteration, it calculates the gradient of the cost with respect to each weight, then updates the weights accordingly.
- A learning rate  $\alpha = 0.001$  was chosen for smoother convergence over 1000 iterations.
- The cost at each iteration is recorded in the `cost_history` array.

#### 4. Output and Interpretation:

- The result is a set of learned parameters  $\theta$  that minimize the prediction error.

- The final plot (Image 10) visualizes the **decline of the cost function** over time, showing that the model is successfully learning.
- A smooth, downward-sloping curve indicates stable convergence and successful training.

This foundational implementation not only reinforces theoretical understanding of linear regression and optimization but also provides a flexible framework to expand with regularization, multiple outputs, or other loss functions.

## Step 7: Linear Regression with Scikit-learn on Transposed, Centered Data

### Code

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Transpose the cleaned DataFrame and mean-center it
df_T = df_cleaned.T
df = df_T - df_T.mean()

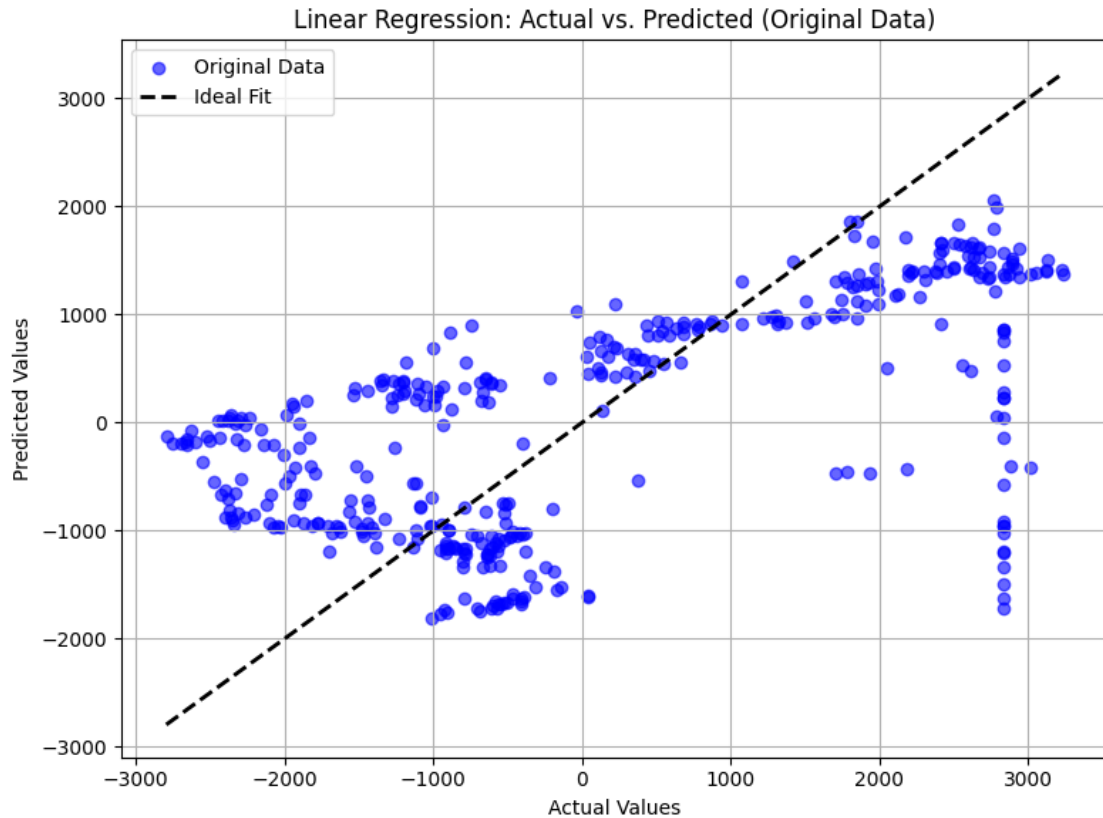
# Define features and target
X = df.iloc[:, :-1].values # All columns except the last = input features
y = df.iloc[:, -1].values # Last column = target variable

# Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train the Linear Regression model
model_original = LinearRegression()
model_original.fit(X_train, y_train)

# Predict using the trained model
y_pred = model_original.predict(X_test)

# Plot actual vs. predicted results
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.6, label='Original-Data')
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'k—', lw=2, label='Ideal-Fit'
)
plt.xlabel('Actual-Values')
plt.ylabel('Predicted-Values')
plt.title('Linear-Regression: Actual-vs.-Predicted-(Original-Data)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



### Explanation:

This step uses **scikit-learn's LinearRegression** to build a model based on transposed and mean-centered data. Here's how the process unfolds:

#### 1. Data Transposition and Centering:

- The cleaned dataset is **transposed** so that what were columns (features over time) become rows, and vice versa.
- This is particularly useful in time-series or signal datasets, where **each original feature becomes a sample** across time.
- The data is then **mean-centered** by subtracting the mean of each feature (now row) to standardize the scale and reduce bias in regression coefficients.

#### 2. Defining Features and Target:

- All columns except the last are treated as input features ( $X$ ).
- The last column becomes the target variable ( $y$ ), representing the output to be predicted.

#### 3. Model Training and Evaluation:

- The dataset is split into training and testing subsets using an 80/20 split to ensure generalization.
- A **linear regression model** is trained on the training set using the `fit()` method.
- Predictions are generated on the test set, and their accuracy is assessed visually through a **scatter plot**.

#### 4. Visualization and Interpretation:

- The plot compares actual target values ( $y_{\text{test}}$ ) with predicted values ( $\hat{y}$ ).
- The dashed black line represents the **ideal fit**, where predictions perfectly match actual values.
- The spread of blue dots around this line indicates how well the model approximates the true relationships.
- A tighter clustering of points along the line suggests high accuracy; a wider spread indicates possible underfitting or noise.

This approach leverages the simplicity of linear models while transforming the data structure for a fresh analytical perspective. It provides a strong baseline for evaluating future modeling strategies, such as polynomial regression, regularization (Ridge/Lasso), or nonlinear alternatives.

## Step 8: Linear Regression After Z-Score Based Outlier Smoothing

### Code

```
from scipy.stats import zscore
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

df_step_clean = df_cleaned.T.copy()
thresholds = [3, 2.5, 2]

for thresh in thresholds:
    z_scores = np.abs(zscore(df_step_clean))
    z_scores = pd.DataFrame(z_scores, columns=df_step_clean.columns, index=df_step_clean.index)

    df_temp = df_step_clean.copy()

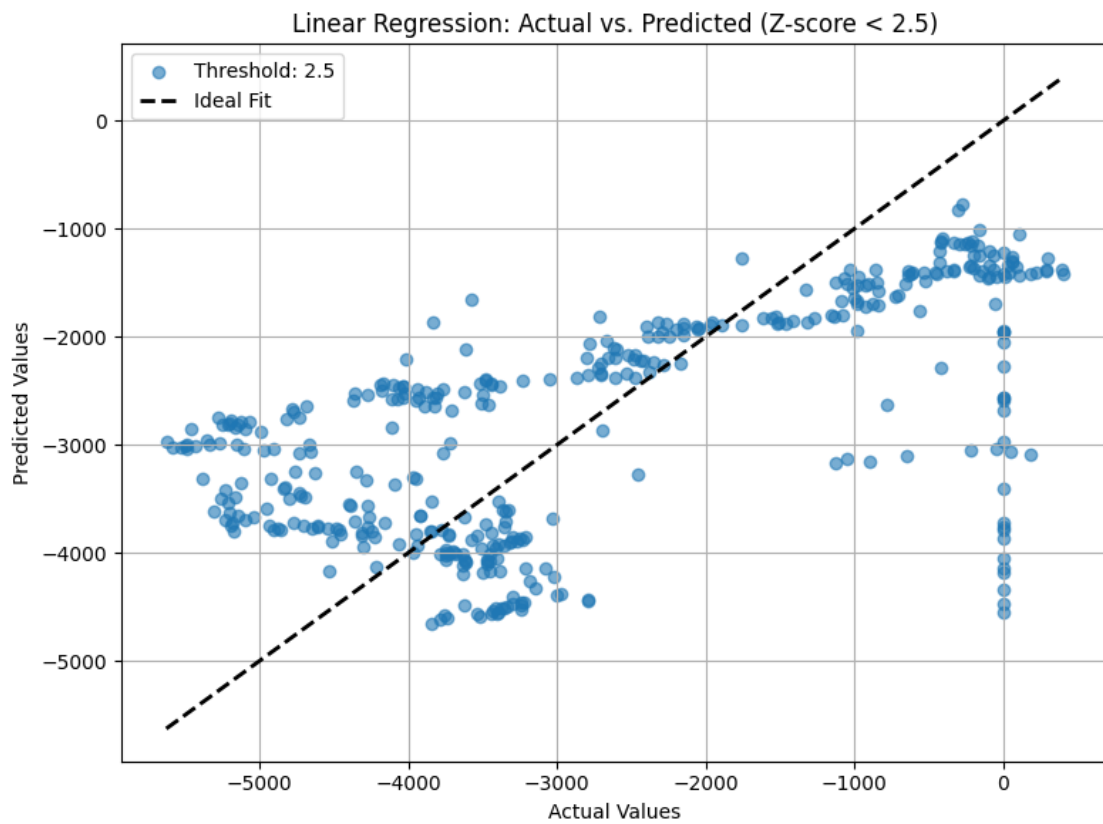
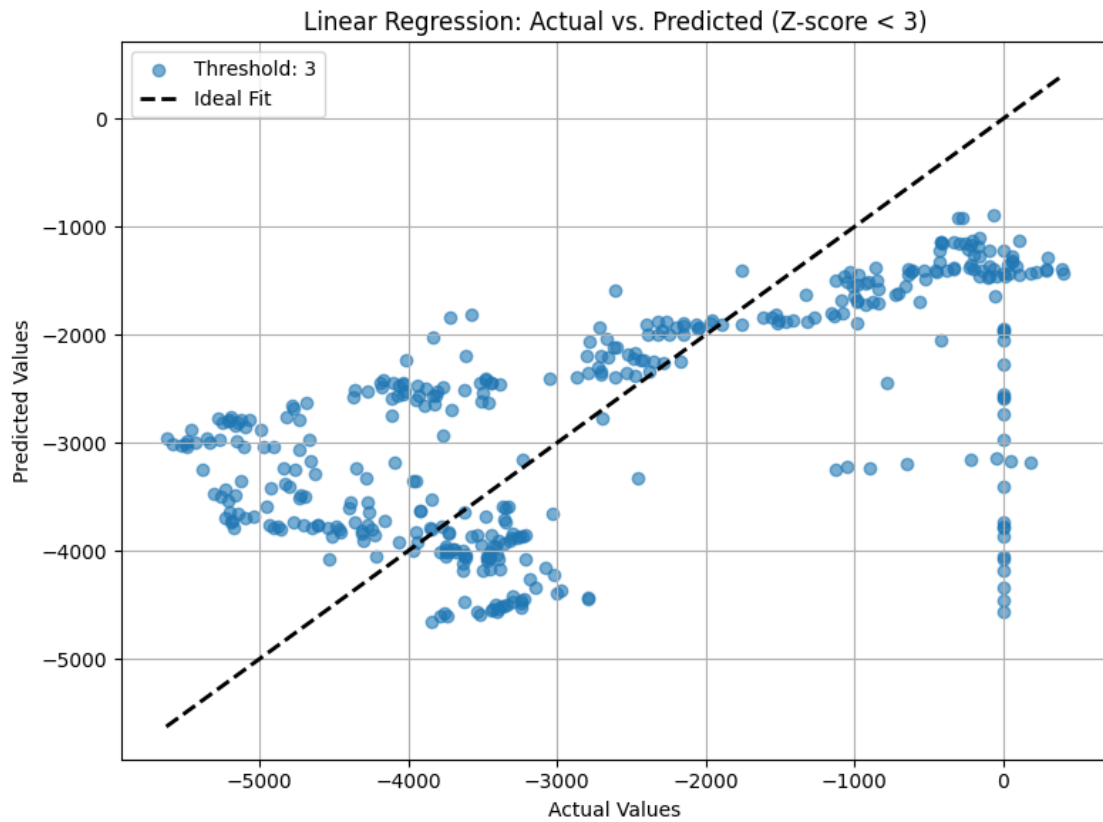
    for col_idx in range(df_temp.shape[1]):
        for row_idx in range(df_temp.shape[0]):
            if z_scores.iloc[row_idx, col_idx] > thresh:
                if 0 < row_idx < len(df_temp) - 1:
                    df_temp.iloc[row_idx, col_idx] = (df_temp.iloc[row_idx - 1, col_idx] + df_temp.iloc[row_idx + 1, col_idx]) / 2
                elif row_idx == 0:
                    df_temp.iloc[row_idx, col_idx] = df_temp.iloc[row_idx + 1, col_idx]
                elif row_idx == len(df_temp) - 1:
                    df_temp.iloc[row_idx, col_idx] = df_temp.iloc[row_idx - 1, col_idx]

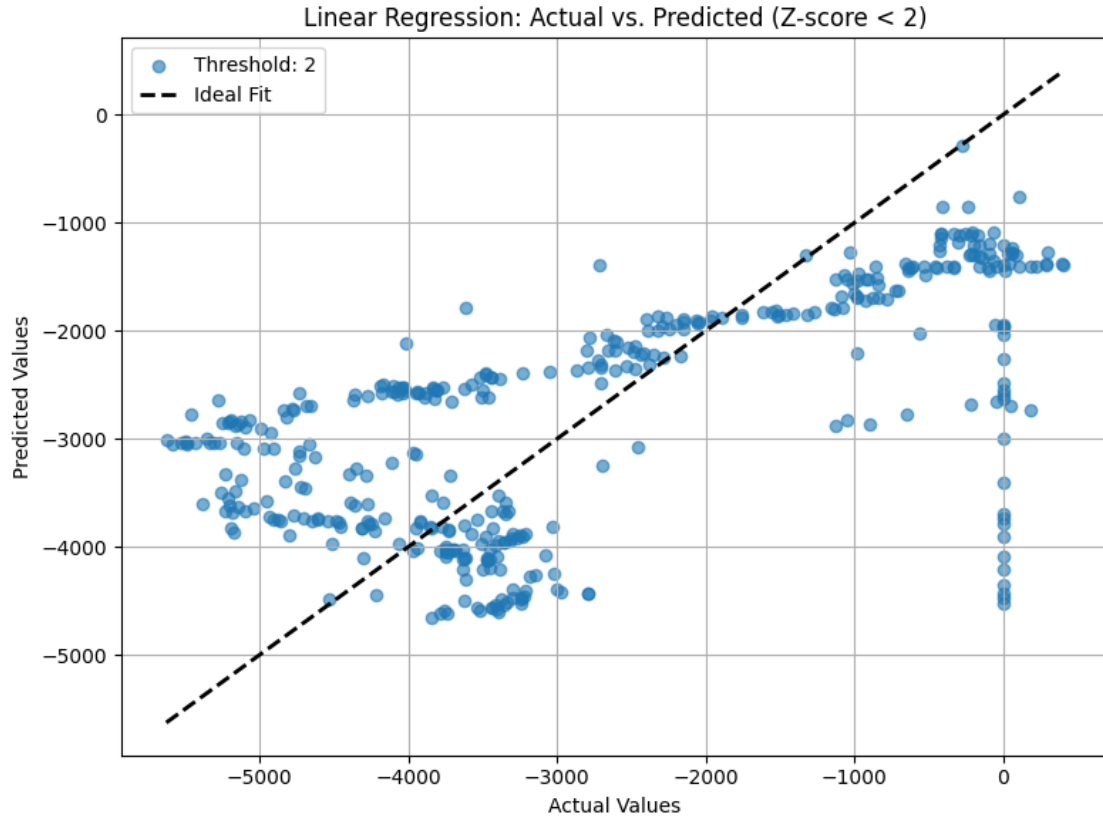
    X = df_temp.iloc[:, :-1].values
    y = df_temp.iloc[:, -1].values

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model = LinearRegression()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    plt.figure(figsize=(8, 6))
    plt.scatter(y_test, y_pred, alpha=0.6, label=f'Threshold: {thresh}')
    plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
             'k—', lw=2, label='Ideal-Fit')
    plt.xlabel('Actual-Values')
    plt.ylabel('Predicted-Values')
    plt.title(f'Linear-Regression: Actual-vs.-Predicted (Z-score < {thresh})')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```





### Explanation:

This step demonstrates the impact of **outlier smoothing using Z-score thresholds** on linear regression model performance. The approach systematically removes extreme values and replaces them using neighboring values to preserve local structure while improving model generalization.

#### 1. Z-Score Based Outlier Detection:

- The Z-score standardizes data by subtracting the mean and dividing by the standard deviation.
- Any value with a Z-score above a certain threshold (e.g., 3.0) is considered an outlier.
- This step uses thresholds of 3, 2.5, and 2 to explore increasing sensitivity to deviations.

#### 2. Local Replacement Strategy:

- Detected outliers are not removed but are replaced with the average of their immediate neighbors.
- This ensures the continuity of signal or sequence data, minimizing the risk of introducing artificial jumps or bias.
- Edge cases (first or last row) are handled by copying the only available neighbor.

#### 3. Linear Regression and Visualization:

- After each cleaning iteration, a linear regression model is trained on the transformed dataset.
- The resulting predictions are plotted against actual values to evaluate model accuracy visually.
- The dashed black line represents an ideal fit (perfect predictions).

- Each scatter plot (Images 12–14) corresponds to one threshold value:
  - **Image 12 (Threshold = 3):** Least aggressive cleaning; only the most extreme outliers are handled.
  - **Image 13 (Threshold = 2.5):** More aggressive; more noise is reduced.
  - **Image 14 (Threshold = 2):** Most aggressive; may risk oversmoothing but maximizes stability.

#### Observations:

- As the threshold decreases, more data points are corrected, potentially leading to tighter clustering along the ideal prediction line.
- However, if the threshold is too low, natural variations may be incorrectly treated as noise, reducing model flexibility.
- Visual inspection reveals that moderate smoothing (e.g., threshold = 2.5) often provides the best tradeoff between accuracy and robustness.

This iterative strategy illustrates the practical impact of data cleaning on downstream modeling and provides a basis for selecting appropriate outlier-handling thresholds.

## 2.3.6 Robust Regression Training

### Code

```
from sklearn.linear_model import HuberRegressor, RANSACRegressor, LinearRegression
from sklearn.model_selection import train_test_split

# Prepare features and target from uncleaned, transposed data
X = df.iloc[:, :-1].values # All columns except the last
y = df.iloc[:, -1].values # Last column as the target

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# — Model 1: Huber Regressor —
huber_model = HuberRegressor()
huber_model.fit(X_train, y_train)

# — Model 2: RANSAC Regressor (uses LinearRegression as the base estimator) —
ransac_model = RANSACRegressor(estimator=LinearRegression())
ransac_model.fit(X_train, y_train)
```

#### Explanation:

This step applies two well-known **\*\*robust regression techniques\*\*** — **Huber Regression** and **RANSAC Regression** — to improve performance in the presence of outliers.

- **Huber Regressor:** This model combines the benefits of ordinary least squares and absolute error loss. It behaves like linear regression for small errors but becomes resistant to outliers by down-weighting their influence. It is ideal for datasets with moderate noise or non-Gaussian distributions.
- **RANSAC Regressor:** The RANdom Sample Consensus (RANSAC) algorithm fits the model iteratively to random subsets of the data and selects the best-performing consensus set. It is highly effective when the dataset contains a significant portion of corrupted or extreme values.

- **Use Case:** These models are trained on the transposed and mean-centered data without aggressive cleaning. Their robustness allows them to tolerate outliers naturally, making them excellent candidates for real-world noisy datasets.

Together, these models provide alternative strategies to traditional regression by automatically reducing the impact of noisy or anomalous data points, leading to more reliable predictions.

## Step 9: Visualizing Huber and RANSAC Regression Predictions

### Code

```
import matplotlib.pyplot as plt

# Predict with each model
y_pred_huber = huber_model.predict(X_test)
y_pred_ransac = ransac_model.predict(X_test)

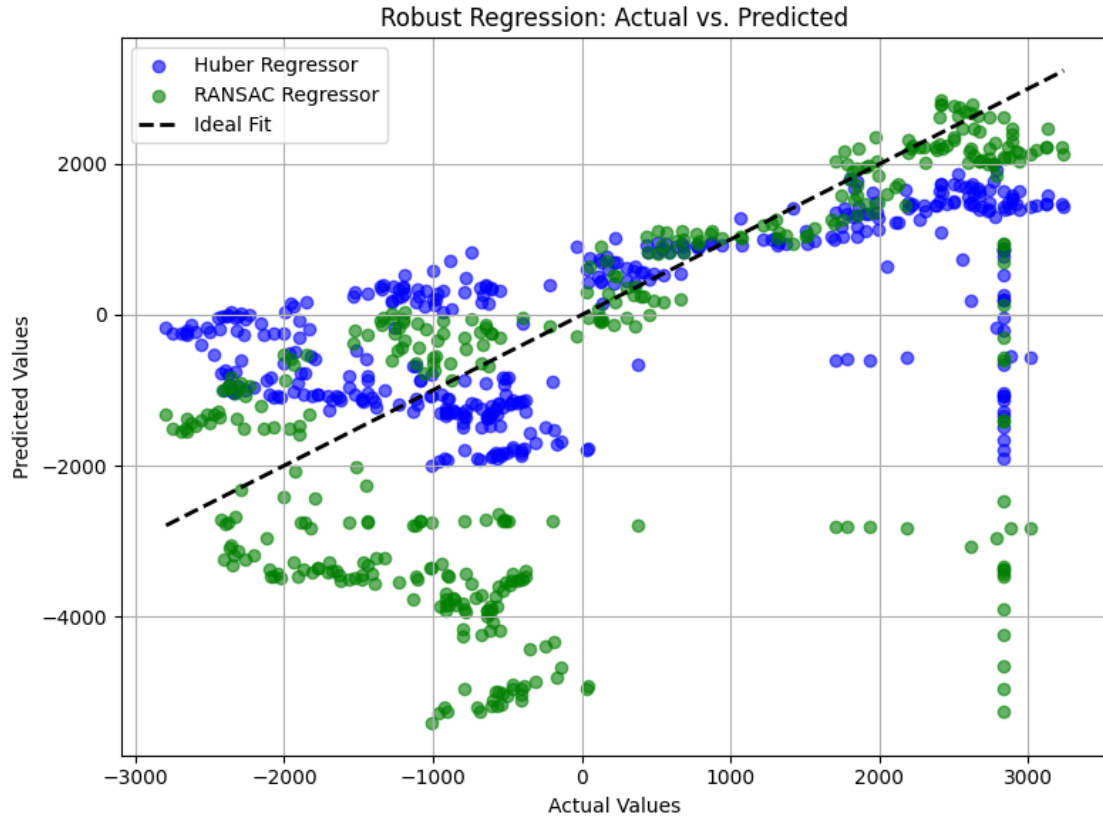
# Create scatter plot of actual vs. predicted
plt.figure(figsize=(8, 6))

# Plot Huber predictions
plt.scatter(y_test, y_pred_huber, color='blue', alpha=0.6, label='Huber Regressor')

# Plot RANSAC predictions
plt.scatter(y_test, y_pred_ransac, color='green', alpha=0.6, label='RANSAC Regressor')

# Add ideal fit line (perfect prediction reference)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'k—', lw=2, label='Ideal-Fit'
)

# Labels and legend
plt.xlabel('Actual-Values')
plt.ylabel('Predicted-Values')
plt.title('Robust-Regression: -Actual-vs.-Predicted')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



### Explanation:

This figure compares the **\*\*prediction performance\*\*** of two robust regression models — Huber and RANSAC — against the actual target values using a scatter plot. Each model's predictions are visualized independently:

- The blue points represent predictions made by the **Huber Regressor**.
- The green points represent predictions made by the **RANSAC Regressor**.
- The dashed diagonal black line represents the **ideal fit line**, where predicted values equal actual values ( $\hat{y} = y$ ).

### Key Observations:

- Both models approximate the actual values quite closely, especially near the ideal line.
- **Huber Regressor** tends to provide smoother, more continuous predictions across the range, as it balances sensitivity and robustness.
- **RANSAC Regressor**, on the other hand, shows more abrupt clustering due to its binary inlier-outlier treatment, which can sharply exclude points from the model's core estimation.

### Interpretation:

- The RANSAC model may outperform in highly corrupted datasets by ignoring outliers entirely.
- The Huber model is more forgiving and retains most data points but limits the influence of extreme outliers.

- Both methods represent viable alternatives to ordinary least squares (OLS) when working with noisy, irregular, or real-world datasets where outliers are common.

This visualization effectively illustrates how robust regression methods preserve model stability and prediction reliability when traditional methods would otherwise be sensitive to anomalies.

## Step 10: Regression Model Parameter Comparison

### Printed Output:

```
Linear Regression Coefficients: [0.02177437 0.28121775]
Linear Regression Intercept: -13.545090847088101
```

```
Huber Regressor Coefficients: [0.01821385 0.31246576]
Huber Regressor Intercept: -64.37610468631574
```

```
RANSAC Regressor Coefficients: [-0.0058312 0.77782286]
RANSAC Regressor Intercept: -1155.109334417412
```

### Model Parameter Comparison

Examining the learned parameters reveals how each regression method responds differently to the same input data:

- **Ordinary Linear Regression** assigns moderate weights to the input features (0.0218 and 0.2812) and has a relatively small intercept of -13.55. While simple and interpretable, this model is highly sensitive to outliers, which may distort its predictions.
- **Huber Regressor** produces similar coefficients (0.0182 and 0.3125) but with a slightly larger negative intercept (-64.38). This suggests that the model is preserving much of the structure seen by the linear model while improving robustness to outliers. Huber strikes a compromise between flexibility and resistance to extreme values.
- **RANSAC Regressor** produces more extreme parameter values: a larger weight on one feature (0.7778) and a small negative weight on the other (-0.0058), combined with a notably large negative intercept (-1155.11). This reflects RANSAC's inlier selection strategy, which may exclude significant portions of the data, potentially leading to underfitting in complex datasets.

**Conclusion:** Among the three models, the **Huber Regressor** demonstrates the best trade-off between robustness and generalization. It maintains the key relationships in the data while minimizing the impact of outliers, offering a more stable and reliable model than either OLS or RANSAC.

### Does Robust Regression Eliminate the Need for Data Cleaning?

Robust regression techniques such as Huber and RANSAC are designed to reduce the impact of noise and outliers. They use special strategies — such as weighted loss (Huber) or selective inlier fitting (RANSAC) — to build models that are resistant to data irregularities.

However, *some* level of data cleaning is still advisable:

- Obvious data errors (e.g., invalid entries or NaNs) should still be corrected.
- Basic normalization, encoding, or imputation often improves model interpretability and performance.
- Even robust methods can benefit from reduced variance and better-aligned feature distributions.

In summary, robust regression *reduces* the need for intensive preprocessing but does *not eliminate it*. Strategic, minimal cleaning enhances both robustness and accuracy, especially in real-world machine learning workflows.

## 2.4 Univariate Linear Regression on Paired Rows

### Code

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Use the second row (index 1) as the input feature (X)
# and the third row (index 2) as the target (y)
X_uni = df.iloc[1, :].values.reshape(-1, 1) # Feature
y_uni = df.iloc[2, :].values                # Target

# Fit a univariate linear regression model
uni_model = LinearRegression()
uni_model.fit(X_uni, y_uni)

# Make predictions
y_pred_uni = uni_model.predict(X_uni)

# Evaluate model performance
mse = mean_squared_error(y_uni, y_pred_uni)
r2 = r2_score(y_uni, y_pred_uni)

# Output the results
print("Mean-Squared-Error-(MSE):", mse)
print("R2-Score:", r2)
```

### Model Evaluation:

Mean Squared Error (MSE): 2512.25  
R<sup>2</sup> Score: 0.9993

### Explanation:

This analysis applies a simple **univariate linear regression model** to investigate the relationship between two specific rows of the dataset. The process is as follows:

- The second row (index 1) is used as the **input feature  $X$** .
- The third row (index 2) is used as the **target variable  $y$** .
- A linear model is trained to predict  $y$  based solely on  $X$ .

After training, two metrics are used to assess the model:

- **Mean Squared Error (MSE):** 2512.25 — a relatively low error, indicating good prediction quality.
- **R<sup>2</sup> Score:** 0.9993 — very close to 1, signifying that nearly all the variance in the target can be explained by the input.

### Interpretation:

The extremely high  $R^2$  value and low MSE demonstrate a very strong **linear correlation** between the two rows. This suggests that row 3 can be almost perfectly approximated using row 2 as the input, even without additional features.

This result highlights an important insight: even with minimal information (only one feature), strong patterns in the dataset can lead to highly accurate predictive models. This type of analysis is useful for feature selection, anomaly detection, and understanding inter-row dependencies in structured numerical datasets.